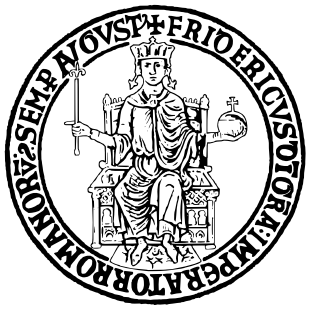
Università degli Studi di Napoli Federico II



Scuola Politecnica e delle Scienze di Base

Dipartimento di Ingegneria Elettrica e Tecnologie dell'Informazione

User guide

Candidato
Giovanni Falcone N97/0000

Contents

Chapter 1

Program structure

```
/
└── hri
    ├── app
    ├── emotion
    ├── robot
    └── util
```

The directory **hri** contains the main application:

- **app**: Contains all files related to the game application, including the Flask server, JavaScript, HTML, and CSS files. It also includes the reinforcement learning algorithm used to generate hints based on the current state of the environment, as well as the set of sentences the robot uses to provide those hints.
- **emotion**: Includes the files responsible for detecting the user's emotional state by capturing and analyzing frames from a webcam.
- **robot**: Manages the robot's behavior and communication, serving as the interface between the software and the robotic hardware.
- **util**: Contains utility files, including a helper script and the global configuration file (config.json).

1.1 The *app* folder

```
/
└── app
    ├── app.py
    ├── play.py
    └── analysis
        ├── analysis.py
        ├── analysis_for_hints.py
        └── general.py
```

```
    ├── psi.py
├── animal_game
├── data
├── flask_utility
├── game
    ├── game.py
    ├── card.py
    ├── player.py
    ├── enivornment.py
├── launch .3 learning
    ├── strategy_condition
        ├── agent_strategy.py
            ├── tom_strategy.py
            ├── external_deception.py
            ├── all other conditions
    ├── agent.py
    ├── learning.py
    ├── qlearning_elements.py
├── sentences
    ├── sentences.py
    ├── conditions
        ├── en
        ├── ita
            ├── tom.json
            ├── no_tom.json
            ├── hidden_deception.json
            ├── superficial_deception.json
            ├── external_deception.json
```

The **app** folder contains:

- **app.py** the ros node which contains the Flask application.
- **play.py** is the main file of Q-learning algorithm.
- **Analysis** contains various scripts to compute general information about different users during the game (turns, received hints, etc.).
- **animal_game** contains the front-end application.
- **data** is the folder in which user data is saved: received hints, clicked cards, etc., and a log file. Each excel file is used by the script of the analysis folder. Depending on how you chose to run the application (whether or not you activated the webcam), you may find two additional files: *filtered.csv* and *full.csv*. These are two CSV files that contain the emotions analyzed during the course of the game.
  - *filtered* contains the user's predominant emotions after the result of a move (i.e., whether a pair was found or not).

  - *full* contains the emotions (as returned by the classifier) frame by frame throughout the entire game.

- **flask_utility** contains utility files for the flask application such as how to handle the user move, a menu to change experimental condition, ...
- **game**: contains the environment logic.
- **launch**: contains the launch file.
- **learning**: contains logic of Q-learning algorithm and how the agents should suggests. The specific strategy used by the agent—i.e., how it should provide hints about the first and second card—is defined in the *strategy_condition* folder.

  Each file in this folder represents a different experimental condition. To add a new condition, simply create a new Python file in the *strategy_condition* directory, ensuring it implements the expected interface defined in *agent_strategy.py*.
- **sentences** contains the sentences that robot should say each time that provide an hint based on experimental condition.

1.2 The emotion module

```
src
├── detection.py
└── emotion_node.py
```

The file *emotion_node.py* contains the ROS node responsible for detecting the user's emotional state based on facial expressions. It accesses the webcam and processes each video frame using the functions defined in *detection.py*.
The *detection.py* file provides the core functionality for:

- Emotion recognition, using the DeepFace library.
- Head pose estimation, using MediaPipe.

1.2.1 The robot module

```
robot
├── launch
└── src
    ├── feedback
    │   ├── emotion_processor.py
    │   ├── feedback.py
    │   └── robot_emotion_controller_.py
    └── gestures
        ├── indefinitesmile_gesture.json
        └── other gestures
```

```
├── interaction
│   └── interaction.py
├── model
│   ├── concrete
│   │   └── fuhrat
│   │       ├── furhat_sdk_app
│   │       ├── automatic_head_movements.py
│   │       └── furhat.py
│   └── interface
│       └── robot_interace.py
├── sentences
│   ├── emotions
│   │   ├── gpt
│   │   │   ├── prompt_ita.txt
│   │   │   └── prompt_eng.txt
│   │   └── ita
│   │       └── friendly.json
│   └── interaction
│       └── ita
│           └── interaction.json
├── emotion_csv.py
├── manager_node.py
└── perception_node.py
```

The robot folder contains:

- **launch**: contains the launch file of the ros nodes (manager, perception and emotion)
- **feedback**: Contains the robot's feedback logic—how it responds to the user's actions and emotional states. Main components:
  - I: Core class for managing feedback, including emotion logging, turn-taking, motivation, and more.
  - I: Handles emotion processing logic, such as determining the dominant emotion based on the number of detected frames.
  - *robot_emotion_controller.py*: Controls the robot's facial expressions and LED lights in response to the user's emotions.
- **interaction**: Manages general interactions between the robot and the user. It contains the *interaction.py* which is responsible for generating speech, greetings, motivational phrases, and other conversational behaviors.
- **model**: Defines the robot class structures and interfaces.
  - *concrete/*: Contains concrete robot implementations (e.g., Furhat). The furhat class can use either the remote API or a Kotlin-based skill

(via HTTP calls), depending on the configuration specified in config.json.

- *interface/*: Defines abstract interfaces that all robot implementations must follow (see robot_interface.py).
- *robot_factory.py*: Factory module for instantiating the appropriate robot implementation based on the config.json.

- **sentences**: Manages the set of phrases the robot can say.
  - interaction folder contains the json of sentences that robot should use for greetings, explain rules, ...
  - The emotion folder contains two subdirectories: gpt and ita. The "*gpt/*" folder includes prompts (in both Italian and English) used to generate motivational sentences via language models. "*ita/*" contains a set of hand-written motivational sentences in Italian (friendly.json).

    The structure is designed to be easily extendable: you can add new feedback modes or support additional languages by including corresponding prompt files or manually written sentences in new subfolders.

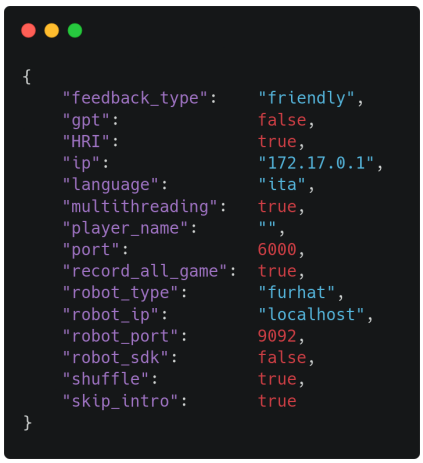
  emotion_sentences.py: Contains the logic for selecting or generating motivational and emotion-based phrases.

- **util** : Contains general-purpose utility functions such as utilities for reading configuration files.
- **emotion_csv.py**: module for handling the storage and management of detected emotions in CSV format.
- **manager_node.py**: The main ROS node that coordinates the interaction between the various modules and the robot. It serves as the central hub for orchestrating the system's behavior.
- **perception_node.py**: a ROS node used to get the user engaged (using robot api).

Chapter 2

How to play

Before running the application you should edit the config.json file



- **feedback_type**: Specifies which set of motivational sentences the robot should use (e.g., "friendly"). This is ignored if gpt is set to true.
- **gpt**: When true, the robot generates motivational sentences using the ChatGPT API instead of predefined phrases.
- **HRI**: If true, the application uses a physical (or virtual) robot for interaction. If false, the interaction is handled without the robot.
- **ip** and **port**: Define the IP address and port of the Flask server running the application.
- **language**: Sets the language for the entire application, including memory tasks, robot interactions, and motivational content. Possible values are "ita" for Italian and "eng" for English.

- **player_name**: The name of the user. This can be left empty since the robot will ask the user for their name during the interaction (unless *skip_intro* is true).
- **record_all_game**: If true, the user's emotional state is recorded continuously throughout the game. If false, emotions are only recorded during feedback moments:
  - In the *e_tom* condition, recording occurs when the robot gives feedback after a move.
  - In other conditions, emotions are recorded for a randomly chosen interval (between 3 to 10 seconds) after each move's result.
- **robot_type**: Specifies the robot model. Currently, only "furhat" is supported.
- **robot_ip**: The IP address of the robot.
- **robot_port**: The port used to communicate with the robot. This is only relevant when the robot used is Furhat and *robot_sdk* is set to true.
- **robot_sdk**: If true, the system communicates with the robot using the Furhat Skill (written in Kotlin) through the specified *robot_port*. If false, it uses the remote API.
- **shuffle**: If true, after several consecutive failed attempts, unmatched card pairs are reshuffled.
- **skip_intro**: If true, the robot skips the introduction phase—it won't introduce itself, ask for the user's name, etc.

2.1 How to start the *robot* application

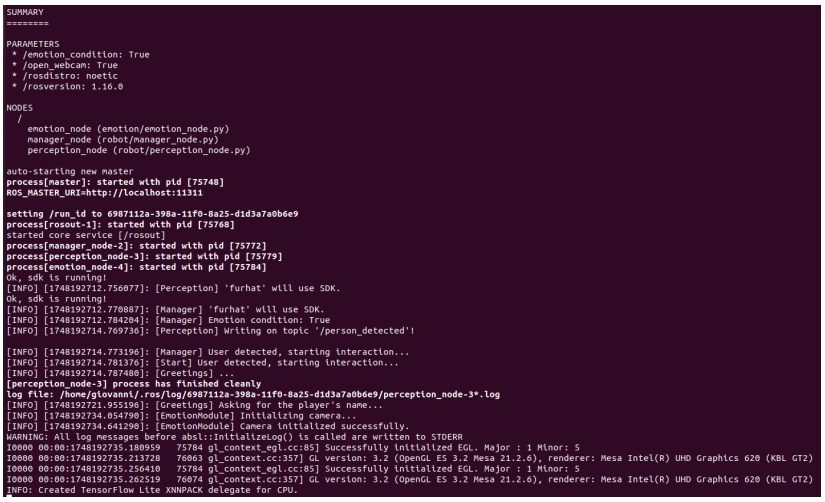
Once you have modified the config.json, you can run the robot application (you can also run app.launch first):

```
roslaunch robot controller.launch emotion_condition:=<value>
                                  open_webcam:=<value>
```

- The `emotion_condition` can either be True or False. If it is True the robot will provide a feedback. The robot won't say anything (to motivate) otherwise.
- The `open_webcam` argument can either be True or False. If it is True, the "emotion_node" node will be executed and, therefore, the webcam will be opened and the user's face analyzed. Otherwise, the node will not be executed. Default is True.

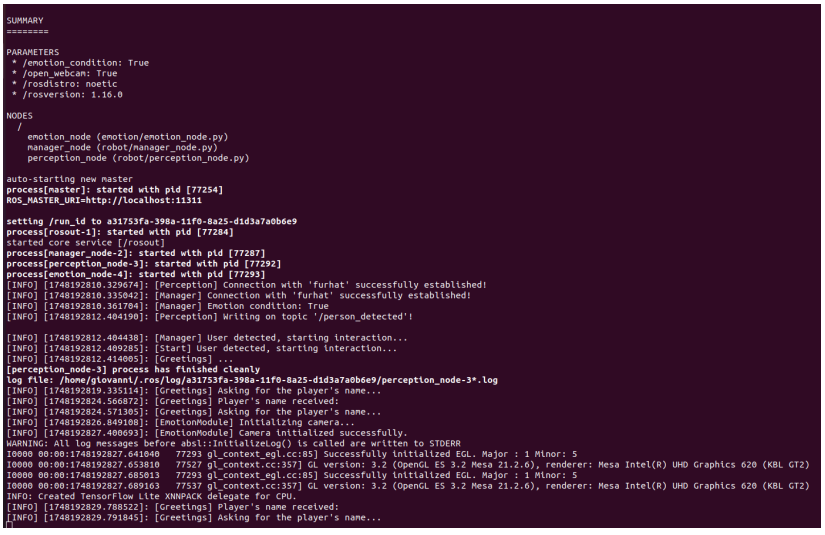
As already said, you can choose to use the sdk or the remote api:

1. If you have chosen to sdk, then you must start the kotlin application first. An exception will be thrown otherwise. Once the sdk is running you have to wait for the user to interact with the robot (i.e., engaged in the interaction). After that, you'll notice the string "server started" on the sdk terminal. Now you can launch the robot application. This is what you'll see:



   Note that the emotion node may take a moment to run.

2. If you have chosen the Remote API, you need to click the 'Remote API' button in the Furhat Launcher, and then launch the robot application.



   In this case, if a user has not engaged yet, you will see the warning `[WARN] : [Perception] Waiting for user....`

2.1.1 Ho to start the *app* application

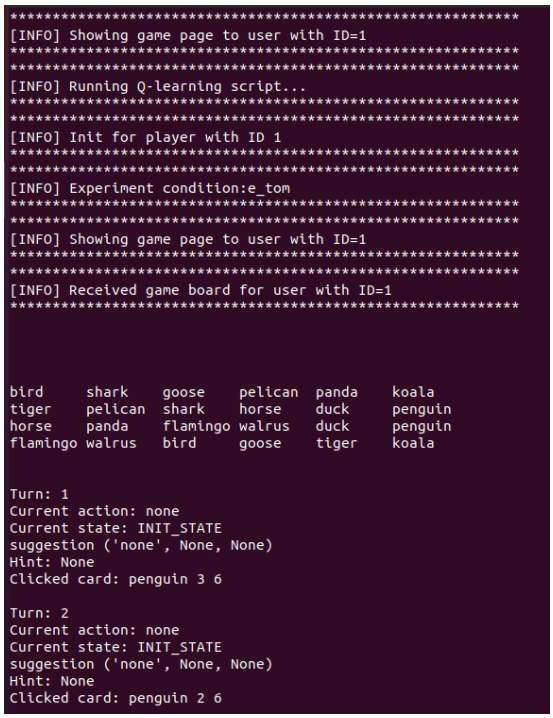
Write on terminal:

```
roslaunch app app.launch id:=<id> condition:=<condition>
```

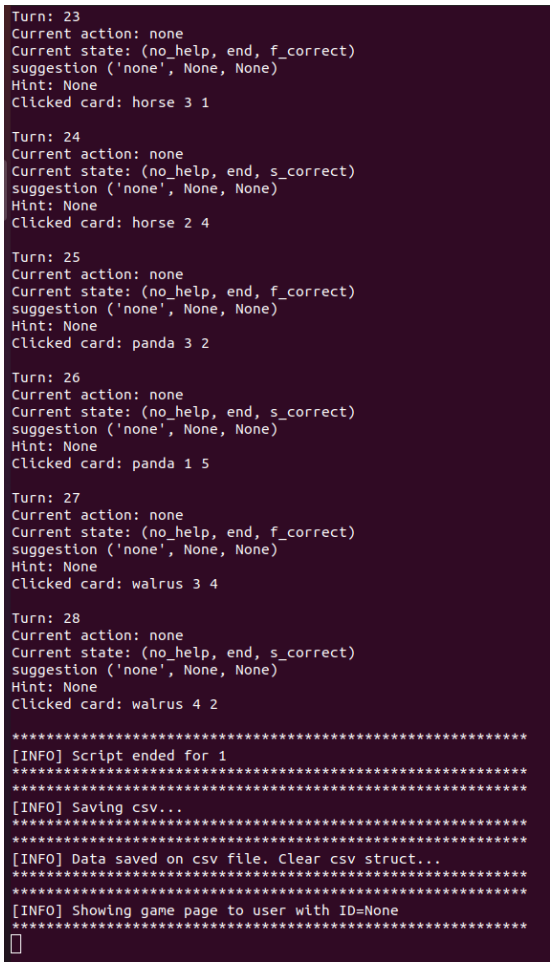
- `id` is the user's id (a directory will be created in the *data* folder). If you write an existing id, you will be asked whether you want to delete the folder with that id or not (if not, you have to restart the application).
- `condition` represents the experimental condition to use:
  - 0: Theory of Mind
  - 1: No-Theory of Mind
  - 2: Deception
  - 3: External
  - 4: Superficial
  - 5: Hidden
  - 6: Emotional Intelligence with ToM

After that, you will see on screen the line `* Running on http://172.17.0.1:5000/ (Press CTRL+C to quit)`. You can now open the web page and and this is what you will see:

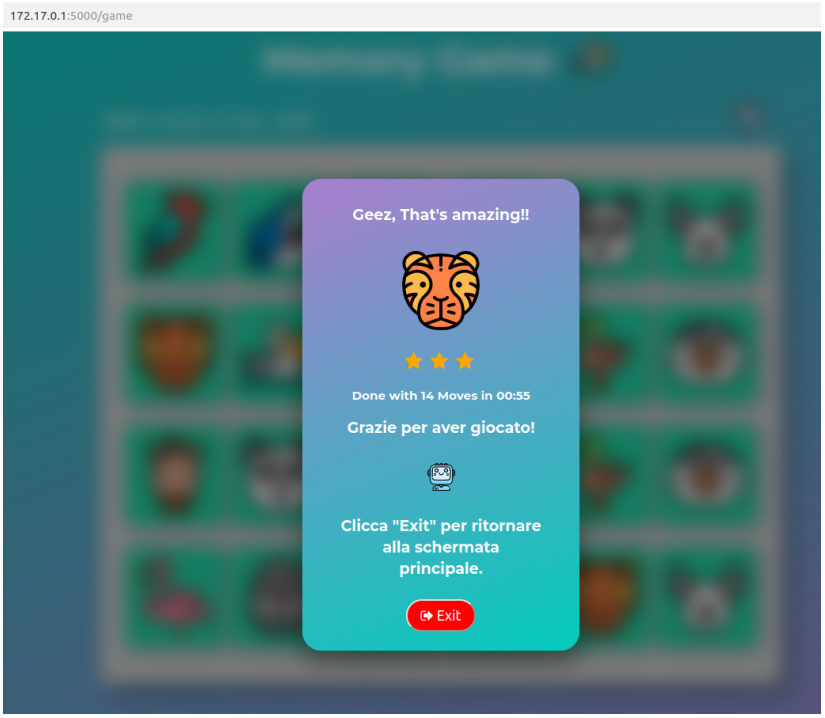
The user can choose the language of the game: this means that the tutorial, pop-ups, etc. will be in English. **To change the language of the robot, remember to edit the config.json file**.
Once the user has seen the tutorial and clicked "*start*" the game screen will be shown and on the terminal you will see the following screen:



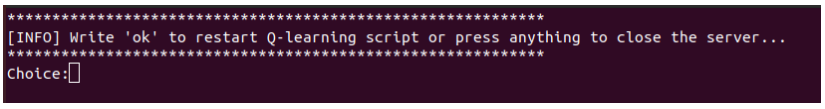
When the user has found all pairs (i.e. the game is finished) then the following screen will be shown:



Therefore, in the "**data**" folder (app/data) you will see the directory "**user_1**" which will contain a txt and csv files.
On the other side, the user will see the following screen:

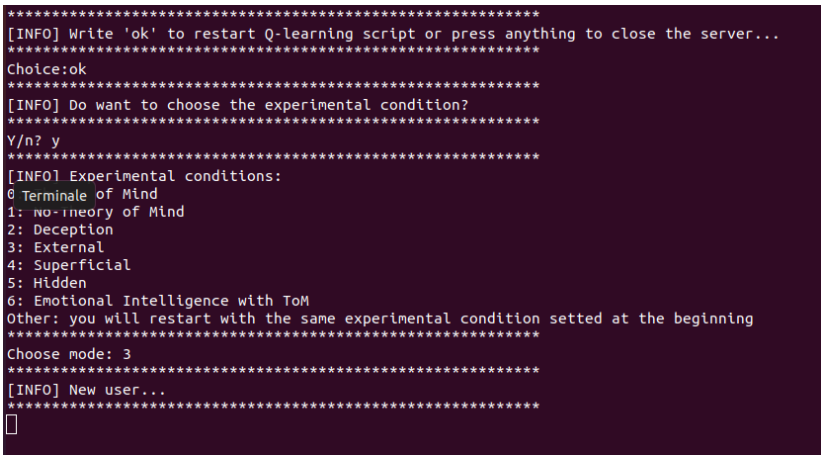


The "**exit**" button must be pressed in order to continue. After that, you'll see the following screen:



If you don't write "*ok*" the server will be shut down (then, press CTRL+C):

If you write "**ok**", a menu will be shown. You can choose a new experimental condition if you wish or continue with the one initially set (or the last one set using the menu).



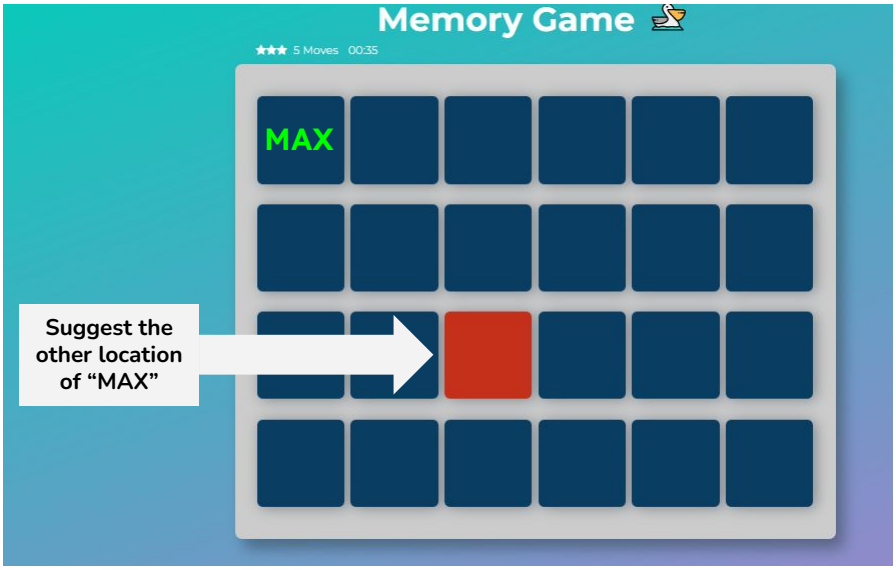
Next, you will see on the web page the initial screen of the game.

Chapter 3

Experimental conditions

Experimental conditions are:

- **ToM - Theory of Mind**: the robot provide hints on the first and second flip motivating its choice



- **NoToM**: the robot provide hints on the first and second flip without motivating its choice (e.g "Try the row/column X"). Moreover, the hint on the first flip is random (i.e the robot will suggest a random card).
- **Deception**: the robot provide hints on both first and second flip. It will use ToM on first flip. Regarding the second flip, there is a 50% that the robot will provide the wrong card.

- **Superficial deception**: the robot will provide hints on the first and second flip by selecting random cards and saying that it is doing so using highly advanced artificial intelligence algorithms.
- **External deception**: the robot will provide hints on the first and second flip. It will use ToM for both. However, regarding the second flip, it will the say the wrong name of the card (the position is correct though).
- **Hidden deception**: the robot will provide hints on the first and second flip. It will use ToM on the first flip. Regarding the second flip, it will suggests the correct card (correct name and position) by saying that it is choosing randomly.
- **E-ToM - Emotional intelligence**: it is the same as ToM, since the emotional part is handled after the robot has provided the suggestion (using ToM).